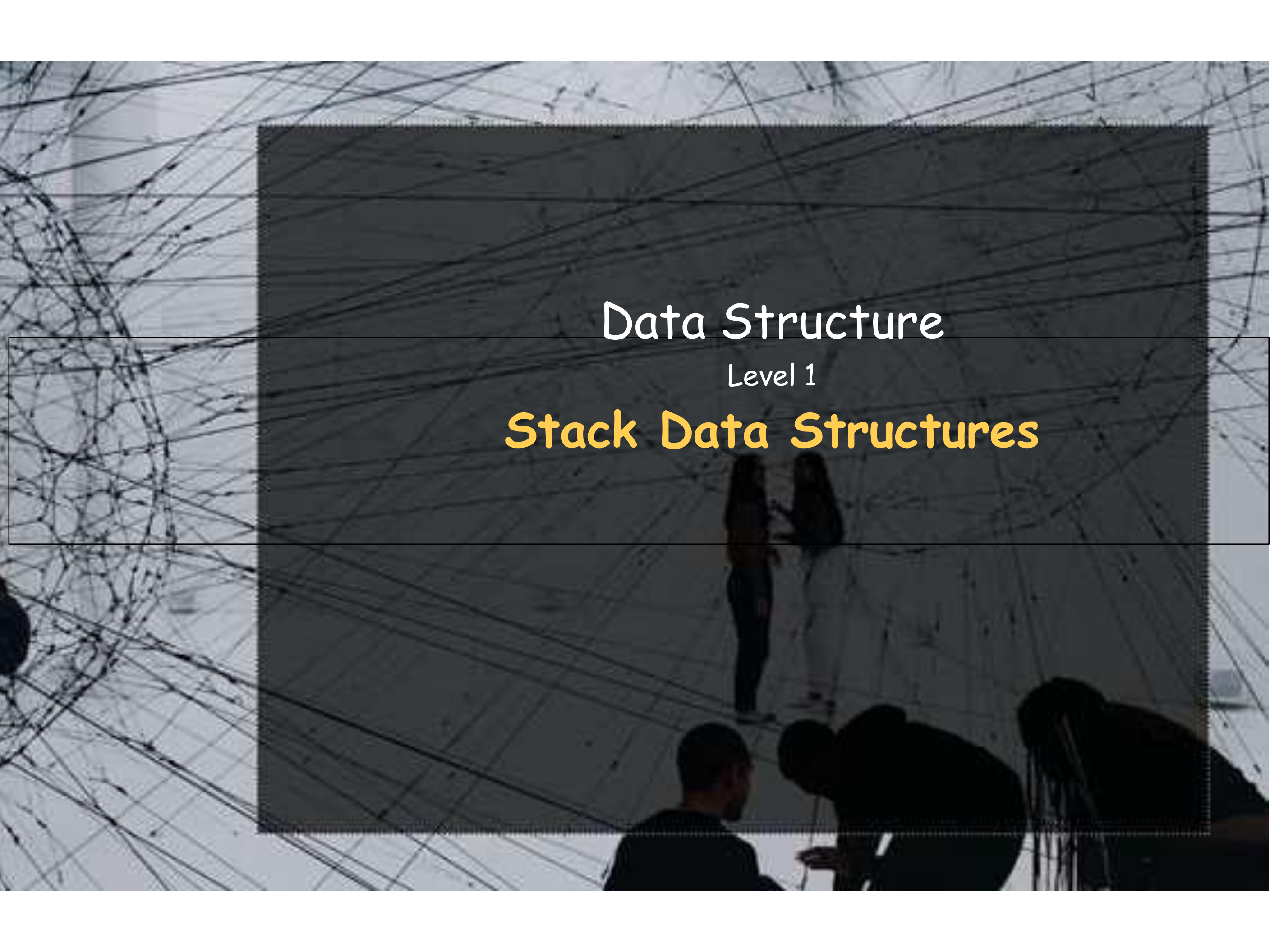


Data Structure

Level 1

Stack Data Structures

The background of the slide features a complex, abstract network of thin, dark lines crisscrossing a light gray background, resembling a web or a neural network. In the lower portion of the image, there are dark silhouettes of several people, some standing and some leaning over, suggesting a collaborative work environment or a meeting. A large, dark gray rectangular area is overlaid on the right side of the image, serving as a backdrop for the text.

Data Structure

Level 1

Stack Data Structures

Definition

A **Stack** is a linear data structure where insertion (push) and deletion (pop) happen only at one end called the **top**.

STL : Standard Template Library
We take stack from it

```
#include <stack>
```

```
Stack <int> stkNumber
```

Explanation

- ◆ In the given code, a stack of integers is created.
- ◆ Numbers are added using push().
- ◆ The program prints and removes numbers using top() and pop() until the stack is empty.

Stack Main Operations

-  `push(x)` → Adds element `x` to top of stack
-  `pop()` → Removes top element
-  `top()` → Returns top element without removing
-  `empty()` → Checks if stack is empty
-  `size()` → Returns number of elements

Example

If we push numbers in this order:

10 → 20 → 30 → 40 → 50

→ Result when popping:

50 → 40 → 30 → 20 → 10

Procedures (Program Flow)

- 1 Include libraries
 <iostream> → input/output
 <stack> → stack container
- 2 Create stack
 stack<int> stkNumbers;
- 3 Insert elements
 Using push()
- 4 Display stack size
 Using size()
- 5 Loop until empty
 Check with empty()
- 6 Print top element
 Using top()
- 7 Remove element
 Using pop()



Note

`top()` only reads the top element.
`pop()` removes it but does not return it.



Comparison

Stack

- ✓ Follows Last In First Out (LIFO) order
- ✓ Insertion and deletion happen at the same end (Top)
- ✓ Used in recursion, undo operations, expression evaluation
- ✓ Access only to the top element

Queue

- ✓ Follows First In First Out (FIFO) order
- ✓ Insertion at rear and deletion from front
- ✓ Used in scheduling, buffering, task processing
- ✓ Access to front and rear elements only

🌟 Summary

- ✓ Stack follows LIFO principle
- ✓ Key functions: push, pop, top, empty, size
- ✓ Used widely in memory management and algorithms

Thank you 😊